



Improve readability of the C++ grammar by adding a syntax for groups and repetitions

Document number:

[P3891R0](#)

Date:

2025-11-22

Audience:

CWG, LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3891/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/concise-grammar.cow



ABSTRACT: Purely editorial changes should be made to the C++ grammar to improve readability, such as adding a syntax for groups and repetitions.

§ Contents

- 1 Introduction
- 2 Motivating example
- 3 Design
 - 3.1 Design constraints
 - 3.2 New syntax
 - 3.3 Alternatives considered
 - 3.3.1 Alternative to “*seq*” and “*seq opt*”
 - 3.3.2 “*seq opt*” vs. “*opt seq*”
 - 3.3.3 Alternative brackets for groups



4	Core wording
4.1	[syntax]
4.2	Bulk operations
4.3	[lex.name]
4.4	[lex.icon]
4.5	[lex.fcon]
4.6	[lex.string]
4.7	[dcl.spec.general]
4.8	[dcl.type.general]
4.9	[dcl.decl.general]
4.10	[dcl.fct]
4.11	[dcl.init.general]
4.12	[dcl.enum]
4.13	[namespace.udecl]
4.14	[dcl.attr.grammar]
4.15	[stmt.select.general]
4.16	[stmt.if]
4.17	[dcl.pre]
5	Library wording
5.1	[locale.numpunct.general]
5.2	[locale.moneypunct.general]
5.3	[format.string.general]
5.4	[time.format]
5.5	[fs.path.generic]
6	References

§ 1. Introduction

The current C++ syntax notation as specified in [syntax] and summarized in [gram] has only a handful of features:

- concatenation, such as *pp-number identifier-continue*,
- alternatives or unions, sometimes specified using "one of", and
- optional expansions, such as *long-suffix_{opt}*.

Two notably absent features are grouping and repetition. This leads to many cases of low expressiveness and grammatical bloat, like our many *X-seq* and *X-list* rules:

declaration-seq:
 declaration declaration-seq_{opt}

template-parameter-list:



In plain English, we say:

” A *declaration-seq* is a *declaration* followed by an optional *declaration-seq*.
A *template-parameter-list* is either a single *template-parameter* or a *template-parameter-list*, followed by a comma token, followed by a *template-parameter*.

No reasonable person should teach the language syntax in those words, but that is what the grammar says. This means that any reader (or author of grammar changes) has to mentally deobfuscate the grammar into something intuitive, like:

” A *declaration-seq* is one or more *declarations*.
A *template-parameter-list* is one or more *template-parameters*, separated by a comma token.

This proposal adds grouping and repetition, which obsoletes all *X-seq* nonterminals and simplifies the specification in many places.

§ 2. Motivating example

With these new features, more concise grammar is possible:

Before	After
<i>compound-statement</i> : { <i>statement-seq</i> _{opt} <i>label-seq</i> _{opt} } <i>statement-seq</i> : <i>statement</i> <i>statement-seq</i> _{opt} <i>label-seq</i> : <i>label</i> <i>label-seq</i> _{opt}	<i>compound-statement</i> : { <i>statement</i> _{seq opt} <i>label</i> _{seq opt} }
<i>initializer-list</i> : <i>initializer-clause</i> ... _{opt} <i>initializer-list</i> , <i>initializer-clause</i> ... _{opt}	<i>initializer-list</i> : <i>initializer-clause</i> ... _{opt} « , <i>initializer-clause</i> ... _{opt} » _{seq opt}
<i>identifier</i> : <i>identifier-start</i> <i>identifier</i> <i>identifier-continue</i>	<i>identifier</i> : <i>identifier-start</i> <i>identifier-continue</i> _{seq opt}

ⓘ NOTE: See §4. Core wording and §5. Library wording for many more concrete examples.

Notably, the boilerplate *X-seq* rules are eliminated. The amount of recursion necessary is also greatly reduced.

§ 3. Design



§ 3.1. Design constraints

- The new grammar syntax needs to be easily writable both in proposals that contain grammatical changes as well as the standard itself. Note that authors frequently use strikethrough or underline text to indicate insertions and deletions, in addition to color. Those styles should be avoided.
- The C++ grammar already contains all sorts of tokens without any delimiters like quotes, which we probably want to keep that way due to familiarity. This means that any brackets and punctuation may visually conflict with C++ tokens.
- Some users may rely on assistive technology such as screen readers. This means that if font choice is the only distinction between e.g. C++ tokens and the new grammar features, the standard would be inaccessible to those users.

§ 3.2. New syntax

X_{seq}

One or more repetitions of X . This replaces $X\text{-}seq$. The new syntax is similar to $_{opt}$, so it is obviously feasible in the standard draft. Proposal authors can use subscript text. Screen readers would pronounce "seq" uninterrupted (ignoring subscript), which is fine as long as "seq" is *only* used in this operator form.

$X_{seq\ opt}$

Zero or more repetitions of X . This replaces $X\text{-}seq_{opt}$.

« $X\ yyy$ »

Groups X and yyy , which allows applying $_{opt}$, $_{seq}$, and $_{seq\ opt}$ to multiple elements. The characters used here are U+27EA MATHEMATICAL LEFT DOUBLE ANGLE BRACKET and U+27EB MATHEMATICAL RIGHT DOUBLE ANGLE BRACKET. These get pronounced distinct from other brackets by screen readers, are handled well by many fonts, and are sufficiently visually distinct from $($, $[$, $\{$, and $<$. LaTeX packages like MnSymbol provide these characters. Paper authors can copy and paste these Unicode characters, use HTML character references such as `⟪`, or use text editor extensions like [Insert Unicode](#) for typing these.

§ 3.3. Alternatives considered

§ 3.3.1. Alternative to “ $_{seq}$ ” and “ $_{seq\ opt}$ ”

An alternatively syntax to $_{seq}$ and $_{seq\ opt}$ briefly considered was superscript “+” and superscript “□” for one-or-more and zero-or-more repetitions, respectively. These characters are widely used in regular expression with this meaning, and superscript asterisks denote the [Kleene Star](#) in computer science papers. However, these characters can be hard to distinguish based on font weight and font family. Replacing $X\text{-}seq$ with X_{seq} also feels like a more natural transition for

C++, and means that existing teaching resource referencing the C++ grammar would be easy to relate to the new format.



§ 3.3.2. “*seq opt*” vs. “*opt seq*”

Both notations are equivalent in the sense that the same inputs would be matched. “*opt seq*” could be argued to be “more natural” because it translates to “optional sequence”.

However, if we consider “*seq*” and “*opt*” to be postfix unary operators, then “ $X_{opt seq}$ ” would be a sequence where every X is individually optional. This is more complex, and the parallel to our existing $X-seq_{opt}$ uses in the grammar is less obvious. It also results in an infinitely ambiguous concrete syntax tree: is the “*declaration_{opt seq}* `int x`; a *declaration* followed by an infinite sequence of absent *declarations*, or is it one absent *declaration*, followed by one *declaration*, followed by an infinite sequence of absent *declarations*? The answer is: it doesn't matter because implementations will figure out how to match “*declaration_{opt seq}*” either way, but we should prefer the syntax that doesn't raise such questions in the first place.

§ 3.3.3. Alternative brackets for groups

There are many possible [Unicode bracket characters](#) that could have been used instead. However,

- some are too exotic, like { S-SHAPED BAG DELIMITERS },
- some are just font variations of existing brackets, like FULLWIDTH or WHITE (which means that users with visual impairment could mistake these too easily, especially with bad choice of font),
- some are used too commonly for mathematical operations, like [CEILING],
- some rely too much on “good fonts” and become too visually similar to parentheses for bold font weight, like (MATHEMATICAL WHITE TORTOISE SHELL BRACKETS),
- etc.

The chosen « MATHEMATICAL DOUBLE ANGLE BRACKETS » do not suffer from any of these issues, although they are visually similar to « DOUBLE ANGLE BRACKETS » used in [Chinese punctuation for proper nouns](#).

§ 4. Core wording

The changes are relative to [\[N5014\]](#).

§ [syntax]

Change [\[syntax\]](#) as follows:



1 In the syntax notation used in this document, ~~syntactic categories~~ non-terminal symbols are indicated by *italic*, *sans-serif* type, and ~~literal words and characters~~

terminal symbols in constant width type. A syntactic element is a terminal symbol, non-terminal symbol, or a group of syntactic elements. A group of syntactic elements is delimited by “<” and “>”. Consecutive syntactic elements are listed from left to right. Alternatives are listed on separate lines except in a few cases where a long set of alternatives is marked by the phrase “one of”. If the text of an alternative is too long to fit on a line, the text is continued on subsequent lines indented from the first one.

2 An optional ~~terminal or non-terminal symbol~~ syntactic element is indicated by the postfix subscript “*opt*”, ~~so~~ .

~~{expression_{opt}}~~

~~indicates an optional expression enclosed in braces.~~

One or more repetitions of a syntactic element are indicated by the postfix subscript “-seq-”.

[Example:

initializer-list:

initializer-clause ...*opt* < , *initializer-clause* ...*opt* > *seq-opt*

This notation means that the non-terminal symbol *initializer-list* is matched by an *initializer-clause*, optionally followed by “...”, followed by zero or more repetitions of “ , *initializer-clause*, and optionally “...”. — *end example*]

2 3 [Note: Names for syntactic categories have generally been chosen according to the following rules:

- *X-name* is a use of an identifier in a context that determines its meaning (e.g., *class-name*, *typedef-name*).
- *X-id* is an identifier with no context-dependent meaning (e.g., *qualified-id*).
- ~~*X-seq* is one or more *Xs* without intervening delimiters (e.g., *declaration-seq* is a sequence of declarations).~~
- *X-list* is one or more *Xs* separated by intervening commas (e.g., *identifier-list* is a sequence of identifiers separated by commas).

— *end note*

§ Bulk operations



IMPORTANT: Please read the editorial notes below. This diff looks small, but it's a massive change with huge implications for the document.

Replace the following non-terminals in the document with X_{seq} :



n-char-sequence *n-char*_{seq}
simple-hexadecimal-digit-sequence *hexadecimal-digit*_{seq}
h-char-sequence *h-char*_{seq}
q-char-sequence *q-char*_{seq}
c-char-sequence *c-char*_{seq}
simple-octal-digit-sequence *octal-digit*_{seq}
s-char-sequence *s-char*_{seq}
r-char-sequence *r-char*_{seq}
d-char-sequence *d-char*_{seq}
declaration-seq *declaration*_{seq}
attribute-specifier-seq *attribute-specifier*_{seq}
function-contract-specifier-seq *function-contract-specifier*_{seq}
lambda-specifier-seq *lambda-specifier*_{seq}
requirement-seq *requirement*_{seq}
statement-seq *statement*_{seq}
label-seq *label*_{seq}
cv-qualifier-seq *cv-qualifier*_{seq}
virt-specifier-seq *virt-specifier*_{seq}
balanced-token-seq *balanced-token*_{seq}
class-property-specifier-seq *class-property-specifier*_{seq}
handler-seq *handler*_{seq}
embed-parameter-seq *embed-parameter*_{seq}
pp-balanced-token-seq *pp-balanced-token*_{seq}

Remove all definitions of the replaced non-terminals. These are all of the form:



X-seq :
*X X-seq*_{opt}



DRAFTING NOTE: This bulk edit results in many places where these syntactic elements are referenced in prose, such as in [\[lex.universal.char\]](#) paragraph 3:



A *universal-character-name* that is a *named-universal-character* designates the corresponding character in the Unicode Standard (chapter 4.8 Name) if the ~~*n-char-sequence*~~ *n-char*_{seq} is equal to its character name [...]

In my opinion, this is acceptable. Referring to *X_{seq}* as a single construct isn't necessarily wrong, even though we've historically tried to reference non-terminal symbols as much as possible. That practice isn't feasible anyway when the grammar is much more powerful and there are far fewer non-terminals.

From an English perspective, both X_{seq} and X_{seq} are pronounced "X sequence" or "X seq.", so no problem is caused.



DRAFTING NOTE: Due to this wording choice, we talk about an X_{seq} as if it still was an X_{seq} non-terminal. For example, we say



[...] from any *declaration* in the ~~*declaration-seq*~~ *declaration_{seq}* of the *translation-unit*.

in [module.global.frag] paragraph 4, rather than considering the *declaration* to belong directly to a *translation-unit*. That is, the middle-man in “*translation-unit* → *declaration-seq* → *declaration*” is eliminated. This would allow using (direct) “of” instead of (indirect) “in”, if we wanted to.

In my opinion, simply applying the bulk edit is fine; pointing out the indirection through $_{seq}$ would be an optional and redundant wording choice, but it wouldn't be incorrect.

Perhaps, CWG could decide on a consistent policy so that we *always* call out the $_{seq}$ indirection, or *never* do so.

§ [lex.name]

Change [lex.name] *identifier* as follows:



identifier:
identifier-start *identifier-continue_{seq-opt}*
~~*identifier* *identifier-continue*~~

§ [lex.icon]

Change [lex.icon] *binary-literal* as follows:



binary-literal:
<one of 0b 0B> *binary-digit* *<'_opt binary-digit>_{seq-opt}*
~~*0b* *binary-digit*~~
~~*binary-literal* *'_opt* *binary-digit*~~

Change [lex.icon] *octal-literal* as follows:



octal-literal:
0 *<'_opt octal-digit>_{seq-opt}*
~~*octal-literal* *'_opt* *octal-digit*~~

Change [lex.icon] *decimal-literal* as follows:



```
decimal-literal:  
  nonzero-digit <['_opt digit]>_seq-opt  
decimal-literal-['_opt digit
```

Do not change [lex.icon] *hexadecimal-literal*:



```
hexadecimal-literal:  
  hexadecimal-prefix hexadecimal-digit-sequence
```

Change [lex.icon] *hexadecimal-digit-sequence* as follows:



```
hexadecimal-digit-sequence:  
  hexadecimal-digit <['_opt hexadecimal-digit]>_seq-opt  
hexadecimal-digit-sequence-['_opt hexadecimal-digit
```



DRAFTING NOTE: *hexadecimal-digit-sequence* is used in so many places that it would be inconvenient to expand it (into *hexadecimal-literal* and other non-terminals). See also [lex.fcon].

§ [lex.fcon]

Change [lex.fcon] *exponent-part* as follows:



```
exponent-part:  
  <[one of e E]> sign_opt digit-sequence  
E-sign_opt digit-sequence
```

Change [lex.fcon] *binary-exponent-part* as follows:



```
binary-exponent-part:  
  <[one of p P]> sign_opt digit-sequence  
P-sign_opt digit-sequence
```

Change [lex.fcon] *digit-sequence* as follows:



```
digit-sequence:  
  digit <['_opt digit]>_seq-opt  
digit-sequence-['_opt digit
```

§ [lex.string]

Change the grammar in [lex.string] as follows:



```
[...]  
raw-string:  
    " d-char-sequence seq opt ( r-char-sequence seq opt d-char-sequence seq opt ) "  
  
r-char-sequence:  
    r-char r-char-sequence opt  
  
r-char:  
    any member of the translation character set, except a U+0029 RIGHT  
    PARENTHESIS followed by the initial d-char-sequence (which may be empty)  
    d-char seq-opt followed by U+0022 QUOTATION MARK  
  
[...]
```



DRAFTING NOTE: There are further changes in this area resulting from the § Bulk operations. The important change in this area is the redefinition of *r-char*.

The current "(which may be empty)" seems incorrect anyway; a *d-char-sequence* cannot be empty, but it is optional. The usual wording is (found in *many* places)

” | The optional *attribute-specifier-seq* [...]

For consistency, we could say

” | [...] by the initial optional *d-seq* *seq* followed by U+0022 QUOTATION MARK

However, "initial optional" feels a bit clunky.

§ [dcl.spec.general]

Change [dcl.spec.general] *decl-specifier-seq* as follows:



```
decl-specifier-seq decl-specifiers-and-attributes:  
    decl-specifier seq attribute-specifier-seq seq opt  
    decl-specifier decl-specifier-seq
```

Replace all occurrences of ~~decl-specifier-seq~~ with decl-specifiers-and-attributes.

§ [dcl.type.general]

Change [dcl.type.general] *type-specifier-seq* as follows:



```
type-specifier-seq type-specifiers-and-attributes:  
    type-specifier seq attribute-specifier-seq seq opt
```



Replace all occurrences of `type-specifier-seq` with `type-specifiers-and-attributes`.

Change [dcl.type.general] *defining-type-specifier-seq* as follows:



```
defining-type-specifier-seq defining-type-specifiers-and-attributes:
    defining-type-specifierseq attribute-specifierseq seq opt
defining-type-specifier type-specifier-seq
```

Replace all occurrences of `defining-type-specifier-seq` with `defining-type-specifiers-and-attributes`.

§ [dcl.decl.general]

Change [dcl.decl.general] *ptr-declarator* as follows:



```
ptr-declarator:
    ptr-operatorseq-opt noptr-declarator
ptr-operator ptr-declarator
```

§ [dcl.fct]

Change [dcl.fct] *parameter-declaration-list* as follows:



```
parameter-declaration-list:
    parameter-declaration ⟨_, parameter-declaration⟩seq-opt
parameter-declaration-list , parameter-declaration
```

§ [dcl.init.general]

Change [dcl.fct] *initializer-list* as follows:



```
initializer-list:
    initializer-clause ... opt ⟨_, initializer-clause ... opt⟩seq-opt
initializer-list , initializer-clause ... opt
```

Change [dcl.fct] *designated-initializer-list* as follows:



```
designated-initializer-list:
    designated-initializer-clause ⟨_, designated-initializer-clause⟩seq-opt
designated-initializer-list , designated-initializer-clause
```

§ [dcl.enum]



Change [dcl.enum] *enumerator-list* as follows:



```
enumerator-list:  
    enumerator-definition ⟨_, enumerator-definition⟩seq-opt  
enumerator-list, enumerator-definition
```

Change [dcl.enum] *enumerator-definition* as follows:



```
enumerator-definition:  
enumerator  
    enumerator ⟨ = constant-expression ⟩opt
```

§ [namespace.udecl]

Change [namespace.udecl] *using-declarator-list* as follows:



```
using-declarator-list:  
    using-declarator ...opt ⟨_, using-declarator ...opt⟩seq-opt  
using-declarator-list, using-declarator ...opt
```

§ [dcl.attr.grammar]

Do not change [dcl.attr.grammar] *attribute-list*:

”

```
attribute-list:  
    attributeopt  
    attribute-list , attributeopt  
    attribute ...  
    attribute-list , attribute ...
```



DRAFTING NOTE: What makes *attribute-list* particularly difficult to change is that each-comma separated element can either be empty, an *attribute*, or *⟨ attribute ... ⟩*, but not just ...

I did not want to factor out a new *attribute-clause* non-terminal, which seems necessary to make *attribute-list* non-recursive in a simple way. We can always make that change later.

Change [dcl.fct] *annotation-list* as follows:



```
annotation-list:
```

annotation ...*opt* <_, *annotation* ...*opt*>_{seq-opt}
~~*annotation-list* , *annotation* ...*opt*~~



§ [stmt.select.general]

Change [stmt.select.general] [selection], [statement] as follows:



```
selection-statement:  
    if constexpropt ( init-statementopt condition ) statement <else statement>  
    opt  
if constexpropt ( -init-statementopt condition - ) statement else statement  
if !opt consteval compound-statement <else statement>opt  
if !opt consteval compound-statement else statement  
switch ( init-statementopt condition ) statement
```

§ [stmt.if]

Change [stmt.if] paragraph 1 as follows:



If the condition ([stmt.pre]) yields **true**, the first substatement is executed. If the **else part statement** of the ~~selection-statement~~ selection-statement is present and the condition yields **false**, the second substatement is executed. If the first substatement is reached via a label, the condition is not evaluated and the second substatement is not executed. ~~In the second form of if-statement (the one including else);~~ In an if statement where the else statement of the selection-statement is present, if the first substatement is also an **if** statement, then that inner **if** statement shall contain an **else part statement**.



DRAFTING NOTE: `if consteval` is not relevant to this change because it can only contain a *compound-statement* as its first substatement.

"Part" is removed because we never define what an "**else part**" is, so this wording smells a bit fishy.

§ [dcl.pre]

Change [dcl.pre] *sb-identifier-list* as follows:



```
sb-identifier-list:  
    sb-identifier <_, sb-identifier>seq-opt  
sb-identifier-list , sb-identifier
```

Change [dcl.pre] *static_assert-declaration* as follows:



```
static_assert-declaration:
    static_assert ( constant-expression <_, static_assert-message>_opt )
static_assert( constant-expression, static_assert-message )
```



§ 5. Library wording

The changes are relative to [N5014].

§ [locale.numpunct.general]

Change [locale.numpunct.general] paragraph 2 as follows:



[...] Integer values have the format:

```
units:
    digits
    digits thousands-sep units
    digit_seq < thousands-sep digit_seq >_seq_opt
digits:
    digit digits_opt
```

and floating-point values have:

```
floatval:
    sign_opt units fractional_opt exponent_opt
    sign_opt decimal-point digits digit_seq exponent_opt
fractional:
    decimal-point digits digit_seq opt
exponent:
    e < one of e E > sign_opt digits digit_seq
e:
    e
    E
```

where the number of digits between *thousands-seps* is as specified by `do_grouping()`. For parsing, if the *digits digit_seq* portion contains no thousands-separators, no grouping constraint is applied.

§ [locale.money.punct.general]

Change [locale.money.punct.general] paragraph 3 as follows:



The format of the numeric monetary value is a decimal number:



value:
 *units fractional*_{opt}
 decimal-point *digits* *adigit*_{seq}
 fractional:
 decimal-point *digits* *adigit*_{seq} *opt*

if `frac_digits()` returns a positive value, or

value:
 units

otherwise. The symbol *decimal-point* indicates the character returned by `decimal_point()`. The other symbols are defined as follows:

units:
 digits
 digits thousands-sep units
 *adigit*_{seq} *⟨ thousands-sep adigit*_{seq} *⟩*_{seq-opt}

In the syntax specification, the symbol *adigit* is any of the values `ct.widen(c)` for `c` in the range `'0'` through `'9'` (inclusive) and `ct` is a reference of type `const ctype<charT>& [...]`

§ [format.string.general]

Change the grammar in [format.string.general] as follows:



positive-integer:
 *nonzero-digit digit*_{seq-opt}
 ~~*positive-integer digit*~~
nonnegative-integer:
 *digit*_{seq}
 ~~*nonnegative-integer digit*~~

§ [time.format]

Do not change [time.format] *chrono-specs*:



chrono-specs:
 conversion-spec
 chrono-specs conversion-spec
 chrono-specs literal-char



DRAFTING NOTE: Even with the new features, this is about as compact as it can be. A *chrono-specs* is matched by a *conversion-spec*, followed by zero or more *conversion-specs*

or *literal-char*. There is no good way to express that on one line.



§ [fs.path.generic]

Change the grammar in [fs.path.generic] as follows:



```
[...]  
relative-path:  
    filename  
    filename directory-separator relative-path  
    an empty path  
    ⟨ filename directory-separator ⟩seq-opt filenameopt
```

filename:
 non-empty sequence of characters other than *directory-separator* characters

directory-separator:
 *preferred-separator directory-separator*_{opt}
 *fallback-separator directory-separator*_{opt}

```
[...]
```



DRAFTING NOTE: A more ambitious change would be to replace *directory-separator* with a new *directory-separator-char*_{seq}. However, *directory-separator* is used *a lot* in subsequent wording, so this would have massive blast radius.

§ 6. References

[N5014] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-08-05)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>